PocketSense/DhanChetna

Progress Report - II

Contents

1 Progress Summary

This document presents the second progress report for the PocketSense (DhanChetna) project. Since the initial report submitted in February 2026, significant progress has been made in implementing the core backend, database layer, and web-based frontend. The project is currently on schedule, with the primary CRUD functionality, authentication, and basic analytics operational.

1.1 Timeline Recap

- **January-February 2026:** Requirements analysis, system design, technology selection (covered in Report I).
- **March 2026:** Database modelling, Django project scaffolding, Firebase authentication integration.
- **April 2026 (current):** REST API development, web frontend implementation, budget and savings modules.

1.2 Key Milestones Achieved

1. Django project structure with modular app layout finalized.
2. PostgreSQL database schema implemented and migrated.
3. Firebase Authentication integrated for both web and API access.
4. Transaction CRUD with soft-delete mechanism operational.
5. Budget and Savings Goal modules implemented.
6. Web frontend with server-side rendering deployed locally.
7. Dark/light theme toggle with CSS custom properties.

2 Implementation Details

2.1 Project Structure

The backend follows Django's recommended app-based modularity:

```
backend/
  apps/
    accounts/      # User model, UserProfile, Firebase auth
    transactions/ # Transaction, Category, IncomeSource
    budgets/       # Budget, SavingsGoal
    analytics/     # Summary, category split, weekly trend
    sync/          # Offline sync push/pull, AuditLog
    advice/        # (stub - planned for next phase)
  pocketsense/     # settings, urls, wsgi
  templates/web/   # server-rendered HTML pages
  static/css/      # design-system stylesheet
```

2.2 Database Implementation

2.2.1 User Management

Two models handle identity and financial context:

- **User** - Custom model with UUID primary key and optional Firebase UID. Replaces Django's built-in `auth.User`.
- **UserProfile** - One-to-one extension storing living situation (hostel / home / rented / PG), income type (pocket money / salary / stipend / freelance), income frequency, and whether food is provided.

2.2.2 Transaction Management

- **Transaction** - Records amount, type (income/expense), category FK, date, optional notes, and an `is_deleted` flag for soft deletes. Financial records are never hard-deleted to preserve audit integrity.
- **Category** - Supports both system-default categories (seeded via management command) and user-created custom categories. Includes an icon field for frontend display.

2.2.3 Financial Planning

- **Budget** - Per-category monthly spending limit. The model exposes a `spent` property that aggregates expenses for the current month via ORM query.
- **SavingsGoal** - Target amount, current amount, and a computed `fill_state` property that returns one of four states (`no_fill`, `less_fill`, `more_fill`, `full_fill`) based on percentage. Auto-marks as completed when target is reached.

2.3 REST API Development

The API is versioned at `/api/v1/` and built with Django REST Framework (DRF).

2.3.1 Implemented Endpoints

2.3.2 Authentication Implementation

Two authentication backends coexist:

1. **Session-based auth** - Used by the web frontend. Login view sets a Django session cookie upon credential verification.
2. **Firebase token auth** - Custom DRF authentication class that validates Firebase ID tokens from the `Authorization` header. Used by the desktop client and any future mobile app.

2.3.3 Soft Delete Pattern

The `DELETE` action on transactions does not remove rows from the database. Instead, it sets `is_deleted = True` and records the timestamp. All list/retrieve queries filter on `is_deleted = False` by default. This preserves a complete audit trail, which is critical for financial data integrity.

Resource	Methods	Status
`/api/v1/transactions/`	GET, POST	
`/api/v1/transactions/{id}/`	GET, PUT, DELETE	
`/api/v1/categories/`	GET, POST	
`/api/v1/budgets/`	GET, POST	
`/api/v1/budgets/status_check/`	GET	
`/api/v1/savings/`	GET, POST	
`/api/v1/savings/{id}/deposit/`	PUT	
`/api/v1/summary/`	GET	
`/api/v1/analytics/category-split/`	GET	
`/api/v1/analytics/weekly/`	GET	
`/api/v1/advice/`	GET	Stub
`/api/v1/sync/push/`	POST	
`/api/v1/sync/pull/`	GET	

Table 1: API endpoint implementation status as of April 2026.

2.4 Web Frontend

The web frontend uses Django server-side templates rather than a separate SPA framework.

2.4.1 Pages Implemented

- **Login page** - Dark-themed with a "Try Demo Account" button for testing.
- **Dashboard** - Four stat cards (today's spending, monthly income, monthly expenses, wallet balance), budget consumption bars, recent transactions list.
- **Transactions list** - Filterable by type, category, and month. Paginated.
- **Add Expense / Add Income** - Forms with category dropdown and date picker.
- **Wallet** - Gradient card showing total income, total expenses, and net balance.
- **Savings** - Goal cards with animated fill bars and deposit functionality.
- **Budgets** - Category budget cards with color-coded progress bars (green $< 80\%$, yellow 80-99%, red $\geq 100\%$).
- **Analytics** - Monthly summary, category pie chart (Chart.js), weekly trend line chart.
- **Settings** - User profile editor for living situation, income type, and food provision.

2.4.2 Design System

A single `style.css` file (800+ lines) implements the full design system using CSS custom properties. Theme toggling swaps the property values and persists the preference in `localStorage`. The stylesheet follows a utility-first approach with semantic class names for component-specific styling.

3 Challenges Encountered

3.1 Firebase Integration with Django

Django REST Framework does not natively support Firebase tokens. A custom authentication backend had to be written, which:

1. Extracts the Bearer token from the request header.
2. Verifies the token against Firebase Admin SDK.
3. Retrieves or creates a local `User` object keyed by Firebase UID.

This required careful handling of the "first login" scenario where a Firebase user exists but no local database record does.

3.2 Dual Authentication Modes

Supporting both session auth (web) and token auth (API/desktop) required middleware adjustments to avoid CSRF conflicts on API routes while maintaining CSRF protection on web form submissions.

3.3 Soft Delete Query Filtering

Ensuring `is_deleted` records are excluded everywhere required overriding the default manager's `get_queryset()` method. Early testing revealed that related queries (e.g., budget spent calculation) were inadvertently including deleted transactions, producing incorrect totals.

4 Testing

4.1 Manual Testing

All CRUD operations have been manually verified through:

- The DRF browsable API (`/api/v1/` in debug mode).
- The web frontend for end-to-end user flows.
- Direct database inspection via Django Admin.

4.2 Seed Data

A management command (`seed_demo_user`) generates realistic demo data including:

- A demo user with hostel student profile.
- 60+ transactions across 3 months with varied categories.
- Pre-configured budgets and savings goals.

5 Pending Work

The following items remain for the final phase (April-May 2026):

1. **Recurring pattern detection** - Implement the rule-based algorithm described in Report I (group by normalized title, detect frequency, flag if interval variance ≤ 2 days).
2. **Expense classifier** - Classify transactions as minor ($\leq$ 10% income), major ($>$ 10%), or minor-repetitive based on frequency.
3. **Advice engine** - Implement the four context-aware rules from the research paper:
   - Food $>$ 45% for hostel students $\rightarrow$ suggest mess plan.
   - Entertainment $>$ 20% $\rightarrow$ review subscriptions.
   - Budget $>$ 90% before 20th of month $\rightarrow$ defer non-essential.
   - ≥ 3 recurring patterns $\rightarrow$ show committed expenses.
4. **CSV export** - One-click download of transaction history.
5. **Desktop application** - Tkinter client with offline sync.
6. **Documentation and final report.**

6 Conclusion

The project is progressing well and is approximately 65% complete. The core data layer, authentication, API, and web frontend are fully operational. The remaining work focuses on the intelligent features (pattern detection, classifier, advice engine) which constitute the primary research contribution of this project. These features are well-defined from the design phase and implementation is expected to be completed within the next 3-4 weeks.